

Day 5: Polymer+Water+Ions Production Simulation with GROMACS and Analysis

This tutorial demonstrates how to perform a **production molecular dynamics (MD) simulation** of a solvated polymer system using **GROMACS**, and how to analyze the results. The workflow includes:

1. **Production MD Simulation:** 5–10 ns simulation at constant pressure (1 bar) and temperature (300 K).
2. **Analysis Stage:** Calculation of key properties:
 - Root Mean Squared Displacement (RMSD) of the polymer chain
 - Radius of Gyration
 - Radial Distribution Function (RDF) for Na-Cl, water-water, and polymer-water (Na-Os)
 - Glass Transition Temperature from simulated annealing

Visualization is performed using **nglview**, **MDAnalysis**, and **matplotlib**.

GROMACS Formats and Models

GROMACS File Formats

- **.gro:** GROMACS coordinate file (atomic positions, box)
- **.top:** Topology file (structure, force field, composition)
- **.tpr:** Binary input file for GROMACS (**gmx grompp**)
- **.edr:** Energy file
- **.trr:** Full-precision trajectory
- **.xtc:** Compressed trajectory
- **.log:** Simulation log

Models Used

- **Force Field:** AMBER03 with TIP3P water
- **Polymer:** Random copolymer of PEO and PET
- **Water:** SPC/E
- **Ions:** Na⁺ and Cl⁻

For more details, see the [GROMACS Lysozyme Tutorial](#).

Step 0: Initialization

Set up the environment for the tutorial. Install required Python packages, create directories, and define reusable functions.

Run this step only once when starting the notebook or after restarting the kernel.

Production Simulations (NPT at 300 K)

Purpose: Perform a production MD simulation at constant pressure (1 bar) and temperature (300 K) to study the polymer system's behavior over time.

Key Parameters:

- Integrator: Leap-frog MD
- Simulation Time: 5–10 ns
- Temperature: 300 K

Example MDP settings:

```
integrator          = md
nsteps              = 2500000      ; 5 ns (or 5000000 for 10 ns)
dt                  = 0.002
tcoupl              = V-rescale
tc-grps             = System
tau_t               = 0.1
ref_t               = 300
pcoupl              = Parrinello-Rahman
pcoupltype          = isotropic
tau_p               = 2.0
ref_p               = 1.0
compressibility      = 4.5e-5
pbc                  = xyz
gen_vel             = no
```

Typical GROMACS commands:

```
gmx grompp -f prod-npt.mdp -c ../eq-npt/eq-npt.gro -p ../topol.top -o
prod-npt.tpr -maxwarn 2
gmx mdrun -s prod-npt.tpr -deffnm prod-npt -v
```

Example SLURM Script for Remote Server

Below is an example SLURM script to run both `gmx grompp` and `gmx mdrun` for a 5 ns production simulation on a SLURM-based cluster:

```
#!/bin/bash
#SBATCH --job-name=gmx_prod
#SBATCH --output=gmx_prod.out
#SBATCH --error=gmx_prod.err
#SBATCH --ntasks=8
#SBATCH --time=06:00:00
#SBATCH --partition=compute
#SBATCH --mem=8G

module load gromacs
```

```
# Step 1: Generate the .tpr file
gmx grompp -f prod.mdp -c ../em/em.gro -p ../topol.top -o prod.tpr -
maxwarn 2

# Step 2: Run production MD
gmx mdrun -s prod.tpr -deffnm prod -nt 8 -v
```

Analysis Stage

Purpose: Analyze the results of the production MD simulation to extract key properties of the polymer system.

Workflow:

1. Root Mean Squared Displacement (RMSD)
2. Radius of Gyration
3. Radial Distribution Function (RDF)
4. Glass Transition Temperature

1. Root Mean Squared Displacement (RMSD) of Polymer Chain

Purpose: Calculate the RMSD of the polymer chain to analyze its structural stability over time.

Procedure:

1. Use **gmx rms** to calculate RMSD:

```
gmx rms -s ../prod-npt/prod-npt.tpr -f ../prod-npt/prod-npt.xtc -o
rmsd.xvg -tu ns -b 0
```

2. Plot RMSD vs time using Python (see notebook for code).

2. Radius of Gyration of Polymer Chain

Purpose: Calculate the radius of gyration to analyze polymer compactness.

Procedure:

1. Use **gmx gyrate** to calculate radius of gyration:

```
gmx gyrate -s ../prod-npt/prod-npt.tpr -f ../prod-npt/prod-npt.xtc -o
gyrate.xvg
```

2. Plot radius of gyration vs time using Python.

3. Radial Distribution Function (RDF)

Purpose: Calculate RDF for Na-Cl, water-water, and polymer-water (Na-Os).

Procedure:

1. Use `gmx rdf` to calculate RDFs:

```
gmx rdf -s ../prod-npt/prod-npt.tpr -f ../prod-npt/prod-npt.xtc -o
rdf_na_cl.xvg -tu ns
gmx rdf -s ../prod-npt/prod-npt.tpr -f ../prod-npt/prod-npt.xtc -o
rdf_na_os.xvg -tu ns
gmx rdf -s ../prod-npt/prod-npt.tpr -f ../prod-npt/prod-npt.xtc -o
rdf_water_water.xvg -tu ns
```

2. Plot all RDFs side by side using Python (see notebook for code).
-

4. Glass Transition Temperature (from Simulated Annealing)

Purpose: Analyze the glass transition temperature from simulated annealing.

Procedure:

1. Prepare a simulated annealing MDP file and run the simulation in a new directory (e.g., `tg-npt`).
2. Extract temperature and density using `gmx energy`:

```
gmx energy -f annealing.edr -o temp_density.xvg
```

3. Fit and plot density vs temperature using Python to estimate T_g (see notebook for code).
-

Visualization

Visualize the polymer structure or trajectory using **nglview** and **MDAnalysis** in Python:

```
visualize_trajectory("polymer_solv_ions.gro")
```

License

This tutorial is provided under the MIT License. Use it freely for educational and research purposes.